

CANLIFAL — AŞAMA B1.11 / MEVCUT UPLOAD KEY ENTEGRASYONU (TEKNİK HAZIRLIK)

Alan	Değer
Rapor tarihi (UTC)	2026-08-11
Kapsam	Sadece teknik hazırlık / doğrulama. Build yok, commit yok, push yok, deployment yok.
Yeni keystore üretildi mi?	HAYIR — hiçbir <code>keytool</code> komutu çalıştırılmadı.
Yeni upload key üretildi mi?	HAYIR
Mevcut imzalama yapılandırması değiştirildi mi?	HAYIR — tek satır kod değiştirilmedi.
Secret değeri yazıldı mı / bastırıldı mı?	HAYIR — hiçbir şifre, alias şifresi, private key veya base64 rapora/log'a yazılmadı.

Bu aşama kullanıcının **orijinal** keystore'u sağlaması için ortamın hazır olup olmadığını doğrular. Anahtar materyali kullanıcı tarafından sağlanmadan hiçbir değer oluşturulmamış veya tahmin edilmemiştir.

1. Mevcut signing configuration incelemesi

Dosya: `mobile/android/app/build.gradle.kts`

İmzalama iki kaynaktan beslenecek şekilde tasarlanmış (`ensureReleaseKeystoreConfigured()`):

- Yerel yol:** `rootProject.file("key.properties")` mevcutsa doğrudan yüklenir (`android/key.properties` + `android/app/release.keystore`).
- CI yolu:** `key.properties` yoksa ortam değişkenlerinden okunur:
 - `ANDROID_KEYSTORE_BASE64` → decode edilip `android/app/release.keystore` olarak yazılır
 - `ANDROID_KEYSTORE_PASSWORD`
 - `ANDROID_KEY_ALIAS`
 - `ANDROID_KEY_PASSWORD`
 - Dörtünden herhangi biri boşsa `false` döner → release signingConfig oluşturulmaz.

`signingConfigs.release` **yalnızca** `hasReleaseKeystore == true` iken oluşturulur.

`buildTypes.release` bu config'i yalnızca varsa bağlar. **Debug imzaya geri düşme yoktur.**

Koruma (afterEvaluate): herhangi bir `*Release` (`assemble` / `bundle` / `package`) görevi çalıştığında keystore yoksa `GradleException` fırlatılır. `assembleDebug` / `CodeQL debug` derlemesi bundan etkilenmez.  Bu koruma korunmalı, kaldırılmamalı.

Değerlendirme: Yapı doğru ve eksiksiz. Kod tarafında hiçbir değişiklik gerekmiyor — kullanıcı secret'ları sağladığında derleme otomatik olarak release imzasını kullanacak.

2. android/key.properties format doğrulaması

mobile/android/key.properties.example mevcut ve beklenen 4 alanı içeriyor:

```
storeFile=release.keystore
storePassword=YOUR_STORE_PASSWORD
keyAlias=YOUR_KEY_ALIAS
keyPassword=YOUR_KEY_PASSWORD
```

✅ Format, build.gradle.kts içindeki keystoreProperties["storeFile"]|"storePassword"|"keyAlias"|"keyPassword"] okumalarıyla **birebir uyumlu**. Gerçek key.properties **ABSENT** (beklenen; .gitignore 'da).

3. build.gradle.kts signingConfig yapısı doğrulaması

Kontrol	Sonuç
signingConfigs { create("release") { ... } } koşullu bloku	✅ PRESENT
keyAlias / keyPassword / storeFile / storePassword atamaları	✅ 4/4 PRESENT
buildTypes.release.signingConfig = signing- Configs.getByName("release")	✅ PRESENT (koşullu)
isMinifyEnabled / isShrinkResources / is- Debuggable=false	✅ PRESENT
Debug fallback	✅ YOK (tasarlandığı gibi)
GradleException koruması	✅ PRESENT (afterEvaluate)

4. Alias doğrulaması (canlifal-upload)

Kaynak	Bulgu
build.gradle.kts	Alias sabit kodlanmamış — key.properties veya ANDROID_KEY_ALIAS 'tan dinamik okunuyor (doğru yaklaşım).
Dokümantasyon (docs/ GOOGLE_SIGNIN_FIX_SHA1_TR.md)	ANDROID_KEY_ALIAS = canlifal-upload olarak belgelenmiş.

Sonuç: Beklenen alias adı **PRESENT (belgelenmiş: canlifal-upload)**. Kullanıcı keystore'u sağlarken keyAlias / ANDROID_KEY_ALIAS değerini **canlifal-upload** olarak girmelidir; aksi halde imza eşleşmez.

5. applicationId doğrulaması

Alan	Değer	Sonuç
namespace	com.mesutbyrm.canlifal	✓
applicationId	com.mesutbyrm.canlifal	✓

6. Secret gizliliği

Bu rapora ve hiçbir dosyaya keystore şifresi, alias şifresi, private key veya base64 içerik **yazılmadı**. ✓

7. Secret değerlerinin ekrana bastırılmaması

Tüm komut çıktıları şifre alanları maskelenerek alındı; hiçbir secret değeri ekrana bastırılmadı. ✓

8. Keystore commit edilmedi

Hiçbir keystore/anahtar dosyası oluşturulmadı, dolayısıyla commit de yok. ✓

9. .gitignore kontrolü

mobile/android/.gitignore (satır 9-12):

```
key.properties
**/*.keystore
**/*.jks
KEYSTORE_CREDENTIALS.local.txt
```

✓ Keystore, `key.properties` ve kimlik dosyası versiyon kontrolünden dışlanmış. Kullanıcı keystore'u yerel `android/app/` altına koyduğunda **yanlışlıkla commit'lenmesi engellenir**.

10. GitHub Actions secret isimleri (CI/CD için)

CI workflow'u (`.github/workflows/build-apk.yml` , satır 226-229) şu **4 secret ismini** zaten okuyor — isimler kod ile birebir uyumlu, **ek kod değişikliği gerekmiyor**:

Secret adı	Beklenen içerik (kullanıcı sağlayacak)	Şu anki durum
<code>ANDROID_KEYSTORE_BASE64</code>	<code>release.keystore</code> dosyasının base64'ü	ABSENT (bekleniyor)
<code>ANDROID_KEYSTORE_PASSWORD</code>	Keystore şifresi	ABSENT (bekleniyor)
<code>ANDROID_KEY_ALIAS</code>	<code>canlifal-upload</code>	ABSENT (bekleniyor)
<code>ANDROID_KEY_PASSWORD</code>	Anahtar şifresi	ABSENT (bekleniyor)

Bu değerler **kullanıcı tarafından** GitHub → Settings → Secrets and variables → Actions altına girilecek. Bu ajan tarafından oluşturulmayacak veya tahmin edilmeyecek. Yerel derleme için alternatif: `android/key.properties` + `android/app/release.keystore` dosyalarını elle yerleştirmek (ikisi de `.gitignore` 'da).

Play Store / Upload Key durumu

Soru	Durum	Neden
Play App Signing aktif mi?	UNKNOWN	Bu ortamda Play Console erişimi yok; tahmin yapılmadı.
Gerçek upload key mevcut mu?	UNKNOWN	Kullanıcı orijinal keystore'u sağlayacak; bu ortamda yok. Firebase'de debug dışı bir sertifika parmak izi ve dokümanite edilmiş bir SHA-1 var (B1.10), fakat anahtarın kendisi burada değil.

SONUÇ

SIGNING CONFIG: READY

KEYSTORE:
ABSENT

ALIAS:
PRESENT (belgelenmiş: canlifal-upload)

PLAY APP SIGNING:
UNKNOWN

UPLOAD KEY:
UNKNOWN

Açıklama: Gradle + CI imzalama altyapısı tamamen hazır ve doğru (`SIGNING CONFIG: READY`). Eksik olan tek şey, kullanıcının sağlayacağı **orijinal keystore dosyası ve 3 şifre/alias secret değeri**. Bunlar sağlandığında hiçbir kod değişikliği olmadan release imzası devreye girecektir.

DUR.